PAHPA: Revolutionizing Kubernetes Autoscaling With Integrated Predictive Analytics and Real-Time Monitoring

Junwei Xiao, Fan Yang, Wei Ai, Tao Meng, and Guoqi Xie, *Senior Member, IEEE*

***Abstract*—Kubernetes provides powerful container orchestration features, like auto-scaling, which dynamically adjusts the scale of containerized applications. However, the default auto-scaling mechanism in Kubernetes typically responds to workload changes only after they have occurred, which can result in resource mismatches. While there has been significant research into proactive auto-scaling approaches, most existing solutions struggle to adapt to the constantly evolving characteristics of prediction targets. This paper presents PAHPA, an intelligent, prediction-assisted approach aimed at enhancing Kubernetes' autoscaling capabilities, which integrates an prediction model, SLMD-LightGBM proposed in this paper, featuring a self-updating mechanism for continuous prediction optimization, queueing theory analysis to improve resource allocation decisions, and a correction mechanism that refines prediction metrics using real-time data. A central insight of this paper is that while predictions are crucial, they should not be the sole basis for decisions and must be complemented by real-time monitoring data to ensure robust and adaptive autoscaling. Experimental results demonstrate that PAHPA significantly enhances system stability and reduces service latency, achieving a 9.5% lower Violation Rate (16.3%) than HPA (25.8%) and a 63% reduction in 99th percentile latency (880 ms vs. 2400 ms). This research highlights how combining predictive analytics with real-time monitoring can lead to more effective autoscaling strategies in cloud-native environments.**

***Index Terms*—Auto-scaling, Kubernetes, PAHPA, QoS, SLMD-LightGBM.**

I. Introduction

MICROSERVICE architecture departs traditionally monolithic applications into multiple fine-grained services. Each service can independently implement distinct functions while maintaining interoperability. These services are typically packaged as containers and dynamically managed by a container orchestrator. Among the various orchestration platforms, Kubernetes stands out as the leading solution [1], offering robust capabilities for deploying, scaling, and managing containerized applications [2].

Kubernetes provides robust auto-scaling capabilities, adjusting pod resources based on demand automatically [3]. As the fundamental operational units within the cluster, pods, each of which comprises one or more containers, are dynamically scaled in response to predefined metrics, such as CPU utilization or memory usage thresholds. This mechanism ensures efficient resource allocation in line with real-time workload requirements [4]. Horizontal Pod Autoscaler (HPA), a prominent auto-scaling controller in Kubernetes, adjusts the number of pod replicas based on specified target metrics. However, the default HPA implementation is reactive, scaling pods only after detecting workload changes. This may result in temporary resource mismatches and potential performance degradation, particularly in environments with rapidly fluctuating workloads [5]. To address these limitations, researchers have proposed various proactive autoscaling methods. These approaches utillize time series prediction models to forecast workload changes and proactively scale pods [6]. The predictive method significantly mitigates the issue of delays.

Due to the inherently dynamic nature of time series data, attaining perfect predictive accuracy remains a challenge. A lot of studies and our research indicate that even state-of-the-art predictive models exhibit an error rate exceeding 20% across diverse scenarios. Consequently, contemporary proactive autoscaling methodologies, especially those predicated on singular resource prediction, often inadequately address the complex requirements of Kubernetes clusters. This limitation will be rigorously analyzed in Section VI.

In practice, it is uncommon for researchers to rely solely on a single metric for implementing proactive scaling. Instead, it is customary to employ a multifaceted approach, leveraging multiple metrics or alternative mechanisms. However, not all metrics

TABLE I
Comparison of Decomposition Methods for Micro-Burst Representation

Method	F1-score	Precision	Recall	BER (%)
LMD	**0.720**	**0.708**	**0.732**	24.7
EMD	0.650	0.632	0.668	24.7
STL	0.414	0.384	0.448	**30.4**

Received 27 May 2025; revised 16 March 2026; accepted 15 April 2026. Date of publication 17 April 2026; date of current version 12 June 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 61902436, in part by the HuNan Provincial Natural Science Foundation of China under Grant 2019JJ50996, and in part by the Excellent Youth Project of Hunan Provincial Education Department under Grant 25B0313. *(Corresponding author: Fan Yang.)*

Junwei Xiao, Fan Yang, Wei Ai, and Tao Meng are with the College of Computer Science and Mathematics, Central South University of Forestry and Technology, Hunan 410004, China (e-mail: junvyx@163.com; yangfanf117@hnu.edu.cn; aiwei@hnu.edu.cn; mengtao@hnu.edu.cn).

Guoqi Xie is with the College of Computer Science and Electronic Engineering, Hunan University, Hunan 410082, China (e-mail: xgqman@hnu.edu.cn).

Digital Object Identifier 10.1109/TSC.2026.3685325

1939-1374 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on August 30,2026 at 06:03:08 UTC from IEEE Xplore. Restrictions apply.

are qualified for time series predictions. For example, CPU and memory utilization metrics can be significantly influenced by device performance and configuration parameters. Therefore, alterations in deployed environments or configurations render previously accumulated data for these volatile metrics obsolete, necessitating the retraining of predictive models.

An additional consideration in proactive scaling pertains to the potential underestimation of actual metric values. Such underestimations can result in an insufficient allocation of pods, thereby compromising system performance. To address this concern, this paper advocates for a hybrid scaling strategy that integrates proactive with reactive scaling to rectify prediction discrepancies. By comparing predicted and real-time metric values, the cluster adopts the higher value to determine the optimal number of pods. This methodology not only mitigates prediction inaccuracies but also enhances the stability and Quality of Service (QoS) within the cluster by sustaining an appropriate pod count over extended periods. The efficacy of this hybrid approach will be empirically validated in Section VI.

In this paper, we present, a novel approach that blends the strengths of predictive analytics and reactive scaling. The paper makes the following tangible contributions.

1) We propose a novel prediction model called SLMD-LightGBM, that combines Local Mean Decomposition, LightGBM, and a self-updating mechanism. The model is to achieve better accuracy in metrics prediction for auto scaling.
2) We construct an auto-scaling framework, PAHPA, which balances proactive and reactive scaling strategies, considering both predicted and current metrics while integrating QoS considerations through the application of queueing theory.
3) We evaluate the performance of PAHPA in reducing resource usage while improving performance compared to standard HPA.

The rest of the paper is organized as follows. In Section II we describe the related work on auto-scaling in Kubernetes. In Section III, we demonstrate the proposed model, while Section IV introduces the scaling system architecture. Section V presents the experimental setup, and the corresponding performance analysis is covered in Section VI. Finally, Section VII provides the conclusion and discusses areas for potential improvement.

II. Related Work

Autoscaling in distributed computing environments has evolved from rule-based heuristics in traditional frameworks to more dynamic, prediction-driven approaches in modern orchestration platforms like Kubernetes. This evolution can be examined along two primary technical axes: predictive analytics (the models and algorithms used to forecast workload) and system monitoring (the metrics and observability data that inform scaling decisions). Early systems often relied on static thresholds applied to basic resource metrics (e.g., CPU), while contemporary research increasingly integrates sophisticated time-series forecasting with multi-dimensional, application-aware telemetry. This section reviews the state-of-the-art by contrasting traditional autoscaling paradigms with Kubernetes-native approaches, focusing on advancements in predictive modeling and metric selection.

A. Predictive Analytics for Workload Forecasting

The shift from reactive to proactive scaling hinges on the accuracy and efficiency of prediction models. Research in this area spans from classical statistical methods to modern deep learning architectures, each offering distinct trade-offs between computational cost and forecast precision.

Traditional Frameworks & Statistical Models: In pre-container and early cloud environments, autoscaling was frequently governed by simple threshold rules or lightweight statistical models. Techniques like Autoregressive Integrated Moving Average (ARIMA) were applied to capture periodic trends in resource demand with minimal overhead [7]. While computationally inexpensive, these models often struggle with sudden, non-linear workload bursts, leading to under-provisioning and service degradation. They represent the baseline against which more advanced, Kubernetes-era predictors are often compared.

Machine Learning Enhancements: The adoption of machine learning (ML) marked a significant step forward in prediction quality for dynamic systems. Models such as Support Vector Regression (SVR) [8] and hybrid wavelet-SVM approaches [9] demonstrated improved ability to model complex, non-linear patterns compared to pure statistical methods. Reinforcement learning (RL) has also been explored, with systems learning scaling policies directly from performance feedback (e.g., response time) to meet QoS objectives [10]. These ML methods provided a foundation for more intelligent scaling in platforms like Kubernetes, where workload variability is high.

Deep Learning & Modern Architectures in Kubernetes: With the rise of microservices and Kubernetes, deep learning models have become prominent for capturing intricate temporal dependencies in containerized workloads. Long Short-Term Memory (LSTM) networks and their variants (e.g., bidirectional LSTM) are widely used for container load forecasting due to their proficiency with sequential data [11], [12]. More recently, architectures designed for patched time series, such as PatchTCN [13], have emerged. These models segment workloads into temporal patches, effectively capturing both local patterns and long-range dependencies, which is particularly valuable for mitigating the impact of bursty traffic common in microservice chains. Frameworks like pHPA [5] further leverage graph neural networks to model inter-service dependencies, optimizing resource allocation across a microservice topology. However, the high accuracy of deep models often comes at the cost of substantial computational resources and a risk of overfitting on limited operational data, presenting a practical challenge for real-time, in-cluster deployment.

B. System Monitoring & Metric Selection

The effectiveness of any autoscaling strategy—whether predictive or reactive—is fundamentally constrained by the quality and relevance of the metrics it monitors. The evolution from

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on August 30,2026 at 06:03:08 UTC from IEEE Xplore. Restrictions apply.

infrastructure-centric to application-centric observability parallels the shift from traditional clusters to Kubernetes-native autoscaling.

Infrastructure-Centric Metrics in Traditional Systems: Traditional autoscaling systems predominantly relied on low-level, infrastructure metrics such as CPU and memory utilization [14]. While straightforward to collect and act upon, these metrics provide an indirect, often lagging indicator of actual application performance. Scaling decisions based solely on resource saturation can miss early signs of performance decay or over-provision for inefficient applications.

Application-Centric & Multi-Metric Approaches in Kubernetes: Kubernetes' flexible monitoring ecosystem, facilitated by tools like Prometheus and custom metrics APIs, has spurred research into richer, application-level signals. Network traffic (e.g., requests per second) [15], [16] and direct performance measures like response time or latency [17], [18], [19] have become critical scaling targets. These metrics more directly correlate with user experience and service-level objectives (SLOs). State-of-the-art approaches often advocate for multi-metric autoscaling. For instance, Zhou et al. [20] combine CPU, memory, queries per second (QPS), response time, and custom metrics to drive scaling decisions, leading to more robust performance in complex, multi-service applications. Research has also begun to incorporate business-level metrics like cost [21], aiming for holistic resource optimization. The challenge in this multi-dimensional space lies in managing increased complexity, potential metric conflict, and the operational overhead of collecting and analyzing diverse telemetry streams in real time.

C. Synthesis

In summary, the state-of-the-art in autoscaling demonstrates a clear trajectory: from simple, reactive rules on basic metrics in traditional distributed systems, towards proactive, model-driven policies leveraging deep learning and multi-faceted observability in Kubernetes environments. The most effective contemporary systems, as reflected in the literature, do not treat predictive analytics and advanced monitoring as separate concerns. Instead, they integrate high-fidelity forecasts (from models ranging from tuned SVR to patch-based deep neural networks) with a curated set of infrastructure and application metrics. This integration enables more anticipatory and precise resource allocation, improving QoS while contending with the inherent trade-offs in model complexity and monitoring overhead. The following sections of this paper build upon this integrated perspective, proposing a method that addresses specific gaps in handling workload bursts and metric relevance within Kubernetes HPA.

III. Methodology and Model Design

In cloud environments, predictive models for autoscaling have been extensively studied, each with its own strengths and limitations. To the best of our knowledge, a gap exists in the field of proactive autoscaling is that no existing model dynamically updates itself based on temporal changes. Without self-updating capabilities, predictive models fail to adapt to the evolving patterns and trends in time series data, resulting in outdated predictions that poorly reflect the current state of the system. Consequently, this misalignment between the model and the dynamic nature of autoscaling processes can lead to inefficient resource allocation, increased operational costs, and degraded system performance.

Many time series prediction methods rely on decomposing the series into distinct components.Traditional approaches often seperate time series data into trend, seasonality and a residual part [22][23]. While these methods are straightforward and interpretable, they fall short when applied to complex, real-world data, especially in cloud environments where workloads often exhibit irregular, non-stationary, and locally varying patterns.

To address this limitation, we adopt Local Mean Decomposition (LMD), which adaptively decomposes time series into locally oscillatory components without assuming global stationarity or fixed periodicity—making it particularly well-suited for capturing transient bursts and non-stationary dynamics prevalent in cloud workload traces.

Accordingly, we propose a novel approach called SLMD-LightGBM. Our approach combines LMD and Light Gradient Boosting Machine (LightGBM) with a self-updating mechanism. Fig. 1 is a simple structure for the approch without self-updating.

LMD is a signal decomposition technique that iteratively extracts Product Functions (PFs) from a time series. Each PF represents a specific frequency band and amplitude modulation, enabling a detailed analysis of local variations. Unlike traditional decomposition methods, LMD focuses on capturing local fluctuations and irregular patterns, making it particularly suitable for cloud workloads [24]. The decomposition process involves the following steps.

First, the original signal is denoted as $NT_0(t)$. Each local extremum point in $NT_0(t)$ will be identified as e_i. We calculate the average value d_i between e_i and e_{i+1} by using (1):

$$d_i = \frac{e_i + e_{i+1}}{2}. \tag{1}$$

To get a local mean function, we draw a straight line between all corresponding extreme points d_i and d_{i+1} and apply a moving average method to smooth the lines, resulting in a local mean function $d_{11}(t)$.

Second, we calculate all of the local magnitudes a_i by using (2):

$$a_i = \frac{|e_i - e_{i+1}|}{2}. \tag{2}$$

The same operation is applied to all a_i as what was adopted to d_i. And an local mean function for a_i is captured and denoted as $a_{11}(t)$, which is also named envelope.

Third, we define a difference value $h_{11}(t)$ by substracting the local mean function $d_{11}(t)$ from the original signal $NT_0(t)$, as shown in (3):

$$h_{11}(t) = NT_0(t) - d_{11}(t). \tag{3}$$

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on August 30,2026 at 06:03:08 UTC from IEEE Xplore. Restrictions apply.

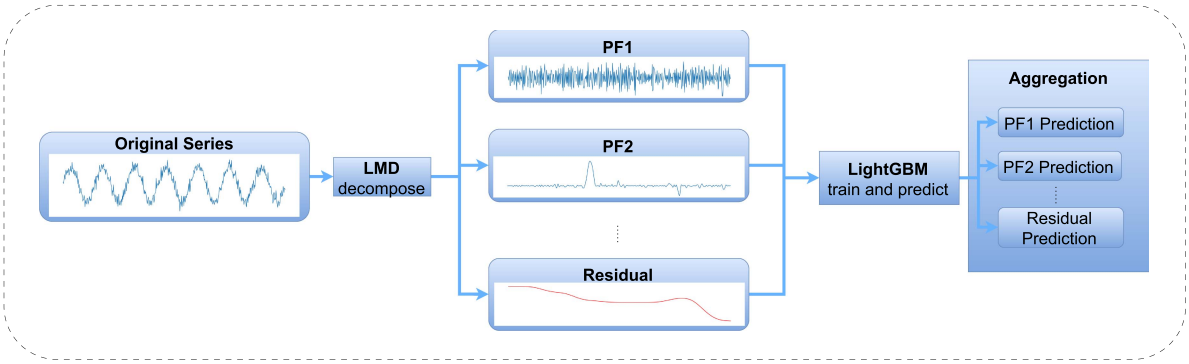


Fig. 1. Combination of LMD with LightGBM.

Then, the signal $h_{11}(t)$ is demodulated by dividing it by $a_{11}(t)$ to get $s_{11}(t)$, as defined in (4):

$$s_{11}(t) = \frac{h_{11}(t)}{a_{11}(t)}. \tag{4}$$

The envelope of $s_{11}(t)$, named $a_{12}(t)$, can be calculated at this point. If $a_{12}(t)$ is equal to 1 constantly, then $s_{11}(t)$ is a pure frequency-modulated signal. If $s_{11}(t)$ does not meet the criteria, we return to (1) and repeat the iteration process on $s_{11}(t)$ until we obtain a pure FM signal $s_{1n}(t)$ by using (5):

$$s_{1n}(t) = \frac{h_{1n}(t)}{a_{1n}(t)}. \tag{5}$$

At each iteration, we compute the demodulated components $h_{11}(t)$, $h_{12}(t)$,..., $h_{1n}(t)$ as defined in (6):

$$\begin{aligned} h_{11}(t) &= NT_0(t) - d_{11}(t) \\ h_{12}(t) &= s_{11}(t) - d_{12}(t) \\ &\vdots \\ h_{1n}(t) &= s_{1(n-1)}(t) - d_{1n}(t). \end{aligned} \tag{6}$$

Then, we multiply all the local magnitude functions generated during the entire iterative process to obtain the final envelope signal $a_1(t)$

$$a_1(t) = a_{11}(t)a_{12}(t)\cdots a_{1n}(t) = \prod_{q=1}^{n} a_{1q}(t). \tag{7}$$

Subsequently, the first product function of the original signal is calculated by multiplying the envelope signal $a_1(t)$ and the pure frequency modulation signal $s_{1n}(t)$, which is defined as (8):

$$PF_1(t) = a_1(t)s_{1n}(t). \tag{8}$$

From the original signal $T(t)$, the $PF_1(t)$ component is separated out as defined in (9), and the resulting new signal $NT_1(t)$ is taken as a new original signal. Eq (1) - (8) are then repeated and the process is iterated j times until $NT_j(t)$ becomes a constant or contains no more oscillations:

$$\begin{aligned} NT_1(t) &= NT_0(t) - PF_1(t) \\ NT_2(t) &= NT_1(t) - PF_2(t) \\ &\vdots \\ NT_j(t) &= NT_{j-1}(t) - PF_j(t). \end{aligned} \tag{9}$$

After the multi-stage decomposition, the original signal $NT_0(t)$ is ultimately expressed as the sum of j PF components and a residual $NT_j(t)$ as (10):

$$T(t) = \sum_{k=1}^{j} PF_k(t) + NT_j(t). \tag{10}$$

This multi-resolution decomposition allows the model to capture both high-frequency and low-frequency components of the time series, providing a more nuanced understanding of the underlying dynamics.

Once the time series is decomposed into PFs, each PF is treated as a separate component for prediction. Here, LightGBM is employed as the predictive engine due to its efficiency and scalability.

LightGBM is an optimized implementation of gradient boosting decision trees, introduced by Microsoft [25]. Its key innovations include a histogram-based algorithm that discretizes continuous features into bins, reducing computational complexity, and a leaf-wise growth strategy that focuses on the leaf with the maximum loss reduction, improving accuracy and efficiency [26]. Notably, studies indicate that for structured and tabular time-series data, tree-based models such as gradient boosting approaches often outperform deep learning models in both efficiency and predictive performance, challenging the notion that deep learning is universally superior [27]. These optimizations make LightGBM particularly well-suited for high-dimensional data and large-scale applications, such as time series prediction. By leveraging LightGBM, the SLMD-LightGBM model achieves fast training speeds and high prediction accuracy [28].

Algorithm 1 delineates the operational workflow of this model. The algorithm begins by iterating through the test_series (line 1-4). For each iteration, it increments a counter to track

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on August 30,2026 at 06:03:08 UTC from IEEE Xplore. Restrictions apply.

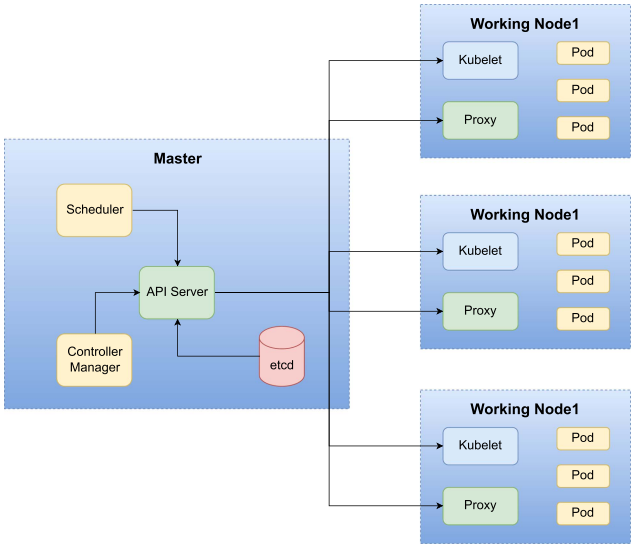


Fig. 2. Basic architecture of Kubernetes.

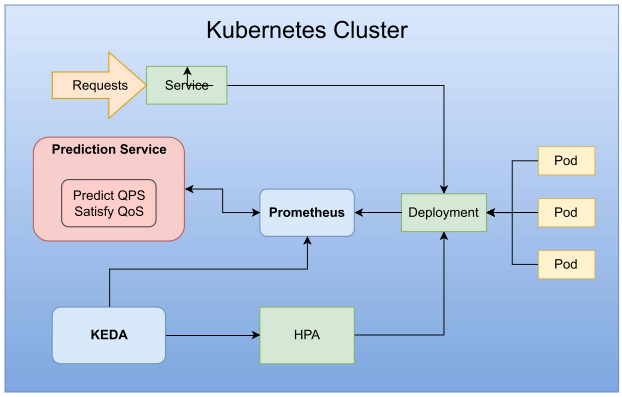


Fig. 3. Key components of PAHPA.

the number of data points processed (line 5). The self-updating mechanism is triggered every $T = 1440$ iterations (lines 6-8), corresponding to a 24-hour period for minute-level data granularity. This cycle is chosen as a pragmatic default for it aligning with common diurnal patterns in real-world business workloads, allowing the model to incorporate a full daily cycle before updating. We note that T is a configurable parameter and can be adjusted based on the observed periodicity of specific workloads.

The updated train_series then undergoes LMD decomposition (line 10), yielding multiple Product Functions (PFs) and a residual part. Each PF represents a distinct frequency component of the time series, arranged from high to low frequency.

The algorithm initiates model training either at the start ($i = 1$) or when the iteration count reaches threshold, 1440 (line 11). During these events, it initializes new training data structures(line 12-13). For each PF, it constructs input-output pairs using a sliding window approach (lines 14-18), trains a LightGBM model on this data (line 19), and appends the trained model parameters to the models list (line 20).

Algorithm 1: SLMD-LightGBM.

Input: $train_series$, $test_series$, $window_size$
Output: $Prediction_{test}$
1: Initialize $count \leftarrow 0$
2: Initialize $models \leftarrow \emptyset$ ▷ An empty list to store trained models
3: Initialize $Prediction_{test} \leftarrow \emptyset$ ▷ An empty list to store predictions
4: **for** $i = 1$ to length$(test_series)$ **do**
5: $count \leftarrow count + 1$
6: **if** $count = 1440$ **then**
7: Append the last 1440 elements of $test_series$ to $train_series$
8: $count \leftarrow 0$
9: **end if**
10: $PFs \leftarrow$ LMD$(train_series)$
11: **if** $i = 1$ or $i \bmod 1440 = 0$ **then**
12: Initialize $X_train \leftarrow \emptyset$
13: Initialize $y_train \leftarrow \emptyset$
14: **for** each PF in PFs **do**
15: **for** $j = 0$ to length$(PF) - window_size$ **do**
16: Append $PF[j : j + window_size]$ to X_train
17: Append $PF[j + window_size]$ to y_train
18: **end for**
19: Train a LightGBM model with X_train and y_train
20: Append the trained model to $models$
21: **end for**
22: **end if**
23: Initialize $yhat \leftarrow 0$
24: **for** index idx, PF in enumerate(PFs) **do**
25: $yhat \leftarrow yhat +$ Forecast$(models[idx], PF)$
26: **end for**
27: Append $yhat$ to $Prediction_{test}$
28: **end for**
29: **return** $Prediction_{test}$

For prediction, the algorithm initializes a prediction variable yhat (line 23). It then iterates through each PF, using the corresponding parameters from the models list to pedict the next value and accumulate these predictions (lines 24-25). The final aggregated prediction is appended to the Prediction_test list (line 27).

This process repeats for each element in the test_series, continuously updating the model and generating predictions. The algorithm concludes by returning the complete list of predictions (line 27). [1] This approach offers several advantages:

1) LMD decomposition enables better capture of multi-scale features in the time series.
2) The use of LightGBM for prediction ensures high accuracy while maintaining computational efficiency.

[1]Code available at: https://gitee.com/junvyxx/slmd-lightgbm

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on August 30,2026 at 06:03:08 UTC from IEEE Xplore. Restrictions apply.

3) The self-updating mechanism allows the model to adapt to changes in data distribution, enhancing long-term predictive performance.

IV. PAHPA Framework

A. Kubernetes Cluster

A typical Kubernetes cluster consists of a master node and multiple worker nodes controlled by the master [29]. Fig. 2 illustrates this basic architecture.

The master node contains etcd, a key-value database storing the system's desired state, controllers that maintain this state, a scheduler managing pod placement, and an API server facilitating communication between the master and worker nodes. In worker nodes, the kubelet ensures pods run as expected, while the kube-proxy manages network communication. Pods are the primary components: as the smallest deployable units in Kubernetes, they encapsulate one or more containers, implement services independently, possess unique IP addresses, and share resources like storage and networking.

B. Horizontal Pod Auto-Scaling

HPA, as a significant controller within Kubernetes cluster, dynamically adjusts the number of pod replicas based on observed metrics such as CPU utilization. A metrics server collects data from pods, which then HPA retrieves via API. The desired number of replicas, denoted as Rep_d, can be calculated as shown in (11):

$$Rep_d = \lceil Rep_{\mathrm{c}} \times \frac{m_{\mathrm{c}}}{m_{\mathrm{d}}} \rceil. \tag{11}$$

Where Rep_d is the desired number of replicas, m_c is the current metric value, and m_d is the desired metric value. The symbol $\lceil . \rceil$ represents the ceiling function, which rounds up the result to the nearest integer. If the desired number differs from the current, the HPA initiates a scaling operation through the controller RC/Deployment.

C. PAHPA (Prediction-Assisted HPA)

PAHPA, our proposed method which is illustrated in Fig. 3, extends the standard HPA by incorporating assisting prediction. Users access application Pods through the Service abstraction in the Kubernetes cluster. When user requests are received, Prometheus continuously collects and stores real-time request metrics, specifically tracking the QPS. The Prediction Service periodically pulls these data from Prometheus and utilizes its forecasting model to predict the required Pod count for the next timestamp, with the prediction results being fed back to Prometheus for storage. Acting as the scaling module, Kubernetes Event-driven Autoscaler(KEDA) regularly retrieves the latest metrics and predictions from Prometheus, then triggers Horizontal Pod Autoscaler (HPA) to execute the actual Pod scaling operations.

The primary metric for PAHPA is QPS, an application-level metric that reflects the real-time demand on the service. While QPS is an application-level concept, it is collected by monitoring traffic at the ingress point of each pod (e.g., via a service mesh sidecar). This pod-level granularity is crucial for two reasons: First, it allows the calculation of the average load per pod (current QPS/number of pods), which is the direct input to the scaling logic (11). Second, it enables the detection of localized issues or imbalances that might be obscured by cluster-wide aggregates. Thus, pod-level monitoring provides the precise data fidelity needed for scaling decisions within the Kubernetes pod abstraction model.

The prediction service containing the predictive model in the cluster retrieves QPS data from Prometheus [30], predicts future QPS, and calculates the required number of pods.

PAHPA adopts queueing theory to measure QoS [31]. When the desired replicas of pods equal to one, the workload is modeled as a M/M/1 queue. An M/M/1 queue is a Markovian queue with a single server, where the arrival process follows a Poisson distribution and the service time follows an exponential distribution. The predicted QPS obtained from the prediction model is treated as the queue arrival rate. The aim is to find the average waiting time per request. The equation is as follows:

$$\bar{W} = \frac{1}{\mu - \lambda} \tag{12}$$

where $\bar{W}$ denotes the average waiting time per request, μ denotes the service rate which represents the maximum QPS the service can handle, and λ is the arrival rate, means the predicted QPS under the given context.

When the desired replicas is more than one. The workload would be seen as a M/M/c queue, indicating the presence of multiple servers. An M/M/c queue is a Markovian queue with multiple servers, where the arrival process follows a Poisson distribution and the service time follows an exponential distribution. It's more complex than M/M/1 queue. The equation to acquire the average waiting time per requests is as follows:

$$\bar{W} = \frac{(c \cdot \rho)^c \cdot \rho}{c! \cdot (1-\rho)^2} \cdot P_0 + \frac{1}{\mu}, \tag{13}$$

where c represents the number of servers, ρ denotes for traffic intensity:

$$\rho = \frac{\lambda}{c \cdot \mu}. \tag{14}$$

P_0 is the probility of being with no QPS:

$$P_0 = 1/\left(\sum_{k=0}^{c-1} \frac{1}{k!} \cdot \left(\frac{\lambda}{\mu}\right)^k + \frac{1}{c!} \cdot \frac{1}{1-\rho} \cdot \left(\frac{\lambda}{\mu}\right)^c \right) \tag{15}$$

By analyzing the average waiting time per request, the predicted replicas of pods can be determined to meet the given QoS requirements.

To mitigate prediction errors—particularly during unpredictable workload bursts—PAHPA adopts a refined hybrid decision strategy that integrates predictive insights with real-time monitoring. Rather than relying solely on predictions, PAHPA evaluates both the predicted and current QPS through a unified queuing-theory model, ensuring robust QoS under varying conditions.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on August 30,2026 at 06:03:08 UTC from IEEE Xplore. Restrictions apply.

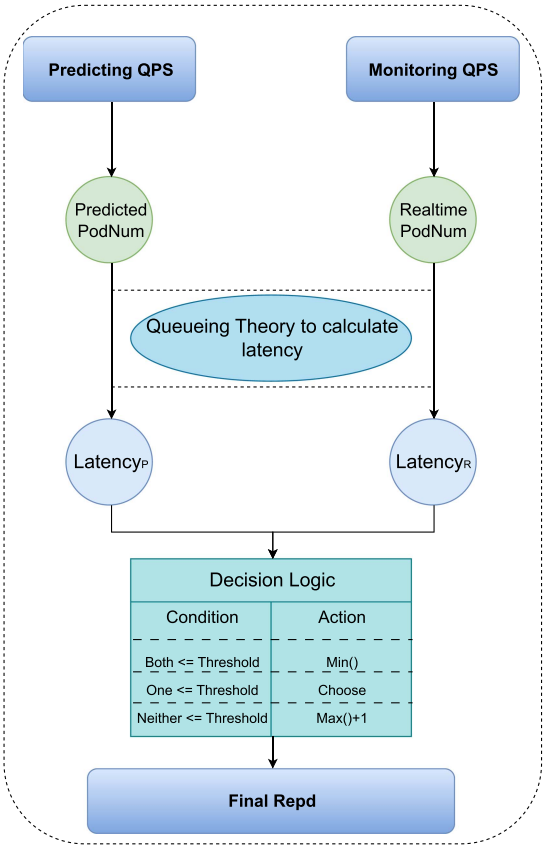


Fig. 4. The process of PAHPA.

As illustrated in Fig. 4, the PAHPA process operates as follows: First, the SLMDLightGBM model forecasts QPS for the next timestamp, which is converted into an initial predicted pod count. Second, the current QPS is measured and likewise transformed into a realtime pod count. Both counts are then independently fed into a queuingtheory (M/M/c) model to compute their corresponding expected waiting times.

The final desired replica count(Rep_d) is determined via a threetier decision rule:

1) Resource-Efficient Mode: If both the predicted and current configurations satisfy the latency target ($\leq$ 1000 ms), the smaller pod count is selected to conserve resources.

2) Safety-Priority Mode: If only one configuration meets the latency requirement, that pod count is chosen to guarantee QoS.

3) Performance-Guarantee Mode: If neither configuration fulfills the latency target, the larger pod count is incremented by one to forcibly meet the performance objective.

This mechanism embodies the core insight of this work: predictions provide valuable guidance but must be complemented—and when necessary, corrected—by realtime monitoring data. By dynamically selecting between proactive and reactive scaling signals, PAHPA achieves a balance between resource efficiency and QoS stability.This intelligent hybrid mechanism embodies the core insight of this work: predictions provide valuable guidance but must be complemented—and when necessary, corrected—by realtime monitoring data. By dynamically selecting between proactive and reactive scaling signals, PAHPA achieves a robust balance between resource efficiency and QoS stability, even under inaccurate predictions or sudden workload changes.

V. Experimental Design

To determine the most suitable decomposition method for capturing micro-burst dynamics in cloud workloads, we conduct a controlled comparison among Local Mean Decomposition (LMD), Empirical Mode Decomposition (EMD), and Seasonal-Trend decomposition using Loess (STL). All methods are applied to the same QPS trace, and their outputs are evaluated based on two criteria: (1) burst energy concentration (BER), and (2) effectiveness in supporting accurate micro-burst detection, measured by F1-score against ground-truth bursts identified via peak analysis.

The results, summarized in Table I, show that while STL achieves the highest BER (30.4%), its F1-score is substantially lower (0.414), indicating poor discriminability between true bursts and noise. In contrast, LMD attains the highest F1-score (0.720) with competitive BER (24.7%), demonstrating its ability to produce a cleaner, more actionable representation of transient events. Based on this evidence, we adopt LMD as the decomposition backbone in our prediction pipeline.

A. Prediction Phase

1) Dataset One: NASA-HTTP(abbreviated as QPS dataset) is a widely used dataset which consists of 3,461,612 HTTP requests made to the NASA Kennedy Space Center WWW server in Florida. We focus on a four-week period within the dataset. To preprocess the dataset, we aggregate logs occurring within the same minute into a single cumulative record which is illustrated in Fig. 5(a). The dataset is split into training (36,813 records in blue) and testing (2,880 records in orange) sets. We employ sliding windows with a fixed size of six corresponding to the window_size in Algorithm 1, to create input and output sets for training and testing.

To mitigate anomalous fluctuations, we remove the lowest 7% of the data based on probability distribution and exclude values exceeding 200. We replace zero values with one to address the impact of points with no requests on the evaluation. Fig. 5(b) illustrates the preprocessed training set.

2) Dataset Two: Cluster-trace-v2018(abbreviated as CPU dataset), released by Alibaba Group, is a production cluster workload dataset that records resource usage metrics (e.g., CPU, memory, disk I/O, and network I/O) from thousands of machines and millions of tasks in 2018. It provides scheduling information, resource allocation, and utilization data, offering real-world support for research on resource management, task scheduling, and cluster performance optimization. Its multi-dimensionality and large-scale nature make it a key benchmark in distributed systems and cloud computing. In this work, we select the CPU usage data of machine "m_2024". Fig. 6 displays the raw sequence of this dataset. Due to its size, we use the last 20% of data points as the test set and the remaining 80% as the training set. No explicit outlier handling is required due to the dataset's low volatility.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on August 30,2026 at 06:03:08 UTC from IEEE Xplore. Restrictions apply.

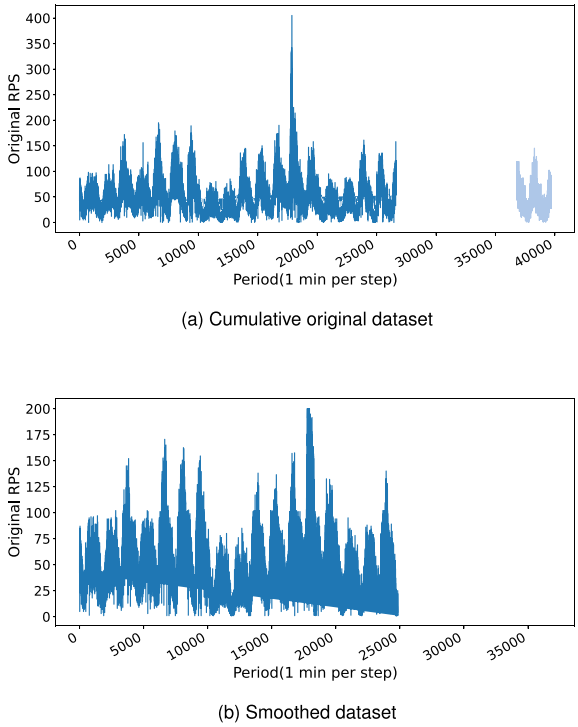


Fig. 5. Comparison of original and smoothed QPS datasets.

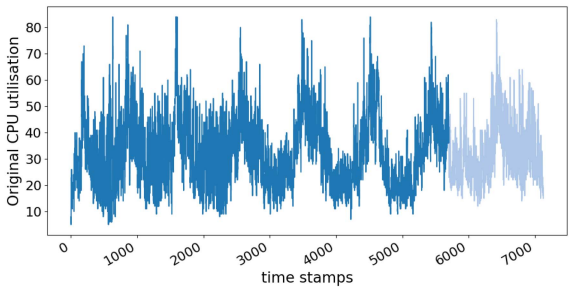


Fig. 6. Original CPU dataset.

3) Evaluation: We use Mean Average Percentage Error(MAPE), Mean Average Error(MAE), Root Mean Square Error(RMSE) and R-squared(R^2) to evaluate the prediction accuracy of the model.

MAPE measures the average percentage difference between predicted and actual values:

$$MAPE = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \times 100, \tag{16}$$

where n is the number of data points, y_i represents the actual value, and $\hat{y}_i$ represents the predicted value.

MAE calculates the average absolute difference:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|. \tag{17}$$

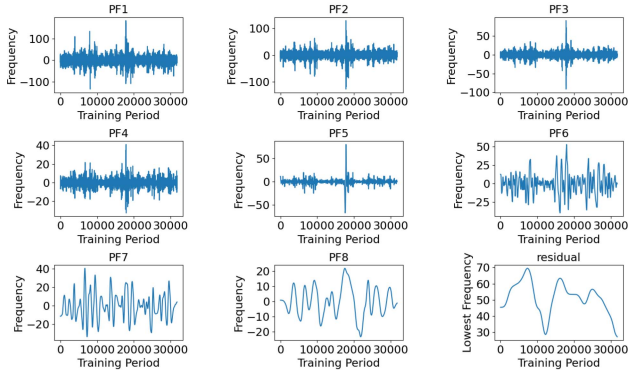


Fig. 7. LMD resulted PFs on QPS dataset.

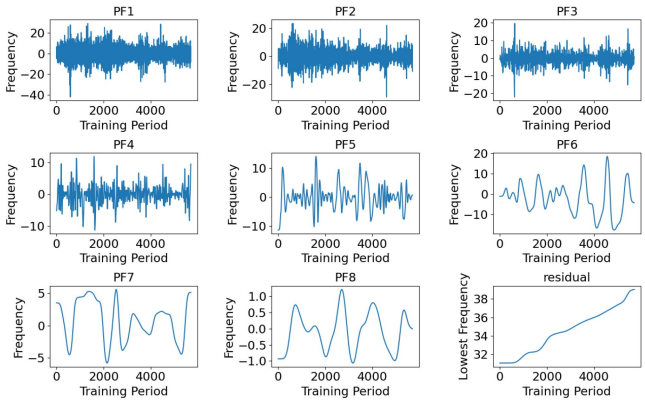


Fig. 8. LMD resulted PFs on CPU dataset.

RMSE is the square root of the average of the squared differences between predicted and actual values:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}. \tag{18}$$

R^2, also known as the coefficient of determination, provides a measure of how well observed series is replicated by the model, with values closer to 1 indicating better fit:

$$R^2 = 1 - \left(\sum_{i=1}^{n}(y_i - \hat{y}_i)^2\right) / \left(\sum_{i=1}^{n}(y_i - \bar{y})^2\right), \tag{19}$$

where $\bar{y}$ represents the mean of the actual values y_i.

B. Scaling Phase

1) Cluster: Our experimental cluster consists of three worker nodes and a master node. We deploy a web application named Podinfo, which provides detailed information about pods in the cluster. Horizontal Pod Autoscaler (HPA) is configured to scale the number of pods based on a threshold of 30 per pod. The qualified average waiting time for each request is set to 1000 milliseconds.

2) Threshold Validation: Load testing experiments validates the scaling threshold. Performance metrics across QPS levels (Table II) show the system maintains good performance up to

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on August 30,2026 at 06:03:08 UTC from IEEE Xplore. Restrictions apply.

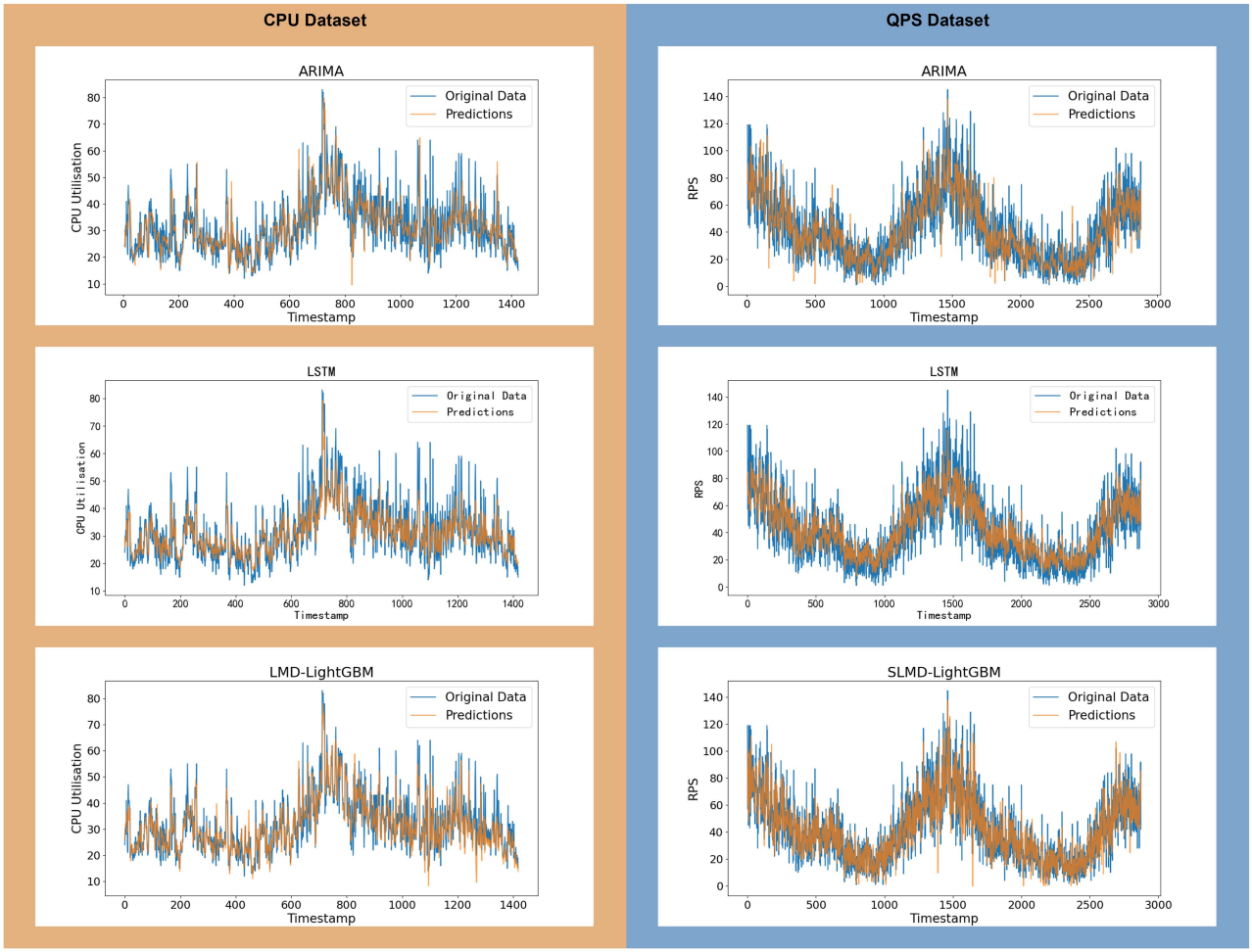


Fig. 9. Prediction results of different datasets.

TABLE II
PERFORMANCE METRICS AT DIFFERENT QPS LEVELS

QPS	Avg Latency (ms)	P95 Latency (ms)
5	149	180
10	129	159
15	115	152
20	138	188
25	133	262
30	315	**703**
35	441	1132
40	997	3362

25 QPS, begins latency degradation at 30 QPS, and experiences severe degradation at 40 QPS.

At 35 QPS, P95 latency reaches 1132 ms, exceeding the 1000 ms target, confirming 30(with 703 ms latency) as the appropriate scaling threshold before performance deteriorates further (3362 ms at 40 QPS).

3) Workload Simulation: To simulate realistic workload conditions, we use Locust [32], a lightweight load testing tool. The traffic rate is derived from the last 1,800 values in QPS dataset, running for a duration of 30 minutes. We adjust the traffic rate from requests per minute to to better match our cluster conditions and demonstrate the adaptability of our model to different time series.

4) Evaluation: There are four metrics used to evaluate the performance of auto-scaling: MaxPod, Cost, Violation Rate (VR) and Latency.

MaxPod refers to the maximum number of pods that exist simultaneously during the workload period, provides insight into the peak resource allocation. Cost represents the total number of pods deployed throughout the workload, reflects the overall resource usage and efficiency of our scaling strategy. VR is the percentage of time during which the QPS per pod falls below 30, indicates how well our system maintains the desired performance threshold. Latency measures the response time at different percentiles, provides a comprehensive view of the system's performance under various load conditions.

VI. PERFORMANCE ANALYSIS

A. Prediction Phase

There are eight profuct functions from high to low frequency and a residual part after the decomposition of the training data by LMD, which is displayed in Fig. 7. The horizontal axis represents timestamps, while the vertical axis shows the

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on August 30,2026 at 06:03:08 UTC from IEEE Xplore. Restrictions apply.

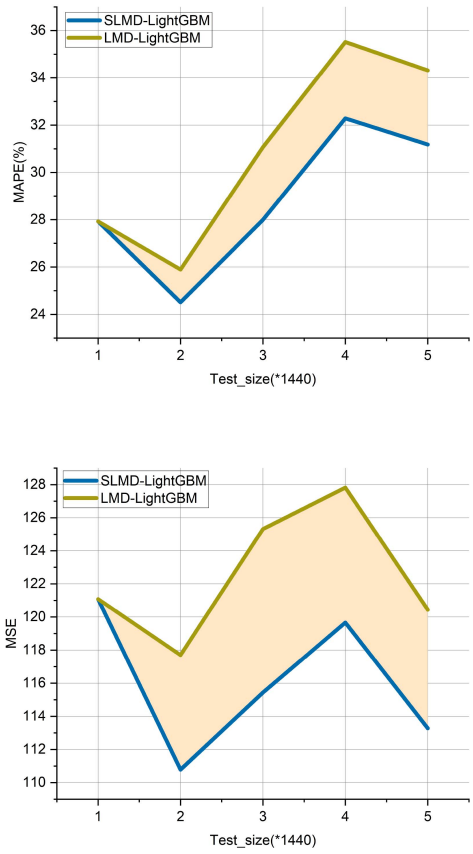


Fig. 10. The comparison of MAPE and MSE between LMD-LightGBM and SLMD-LightGBM on QPS dataset.

decomposed value of the original series. For the CPU dataset, similar LMD decomposition yields eight distinct product functions and a residual component, as illustrated in Fig. 8.

1) CPU Dataset: Given the smaller scale and shorter time span of the CPU utilization dataset, our experiments focus on validating the core LMD decomposition without incorporating the self-updating mechanism. Fig. 9 (CPU Dataset) compares the prediction results of ARIMA, LSTM, and LMD-LightGBM, where blue lines represent the original CPU utilization series and orange lines denote predicted values. The LMD-LightGBM predictions exhibit closer alignment with ground truth in both overall trends and local fluctuations, demonstrating enhanced capability to capture complex temporal patterns.

Quantitative results in Table III (CPU Dataset) confirm this observation, while baseline LightGBM achieves 17.35% MAPE and Bi-LSTM reaches 16.15% MAPE without decomposition, LMD-LightGBM reduces the error to 11.19%, corresponding to an absolute improvement of 6.16 percentage points over standard LightGBM. This performance gain highlights LMD's effectiveness even without self-updating, particularly for stable metrics like CPU utilization where decomposition alone suffices to extract critical temporal features.

2) QPS Dataset: Fig. 9 (QPS Dataset) presents the comparative prediction results of ARIMA, LSTM, and SLMD-LightGBM on the QPS dataset, with blue and orange lines representing the original QPS series and predicted values respectively.

TABLE III
PREDICTION EVALUATION BETWEEN MODELS ON TWO DATASETS

Dataset	Models	MAPE	MAE	RMSE	R^2
QPS Dataset	ARIMA	49.77	13.84	18.26	0.43
	LightGBM	41.72	11.39	14.93	0.62
	LSTM	40.74	10.67	13.62	0.65
	Bi-LSTM	40.46	10.63	13.58	0.66
	PatchTST	40.02	11.38	14.89	0.63
	LMD-LightGBM	25.75	7.71	10.82	0.79
	LMD-LSTM	25.12	7.68	10.62	0.80
	SLMD-LightGBM	24.51	7.59	10.53	0.81
CPU Dataset	ARIMA	18.03	6.05	8.23	0.41
	LightGBM	17.35	5.31	7.42	0.52
	LSTM	17.08	5.16	7.10	0.56
	Bi-LSTM	16.15	4.84	6.46	0.58
	LMD-LightGBM	11.19	3.30	4.43	0.80

The SLMD-LightGBM predictions demonstrate superior alignment with the ground truth, accurately capturing both global trends and local fluctuations of the request rate patterns.

As quantified in Table III (QPS Dataset), conventional models without decomposition exhibit limited accuracy, with Bi-LSTM achieving a MAPE of 40.46%. Even a modern attention-based architecture—specifically, a PatchTST-style Transformer that leverages temporal patching and channel-independent modeling—yields only marginal improvement (MAPE: 40.02%, R^2: 0.63), underscoring the inherent difficulty of forecasting raw, bursty log count series. In contrast, the introduction of LMD (Level-Multi-Decomposition) substantially enhances predictive performance: when applied to decomposed components, both LightGBM and LSTM achieve an absolute MAPE reduction of approximately 15 percentage points. The complete SLMD-LightGBM framework—integrating multi-scale signal decomposition with a self-updating mechanism—attains optimal results with a MAPE of 24.51% and an R^2 of 0.81. This represents a 39.4% relative error reduction compared to the strongest baseline (Bi-LSTM, 40.46% $\rightarrow$ 24.51%), highlighting that effective forecasting of volatile web traffic requires not only powerful sequence modeling (e.g., PatchTST) but, more critically, proper handling of non-stationarity through decomposition.

To confirm the effects of the self-updating mechanism, additional experiments were conducted using different sizes of test data: 1440, 2880, 4320, 5760, and 7220, corresponding to data collected over one, two, three, four, and five days, respectively. Fig. 10 illustrates these results, comparing SLMD-LightGBM with LMD-LightGBM on MAPE and MSE.

SLMD-LightGBM consistently shows better performance, with MAPE values between 24.51% and 32.28% and MSE between 110.78 and 119.66. At day one, there is no difference between two models as the self-updating module is not yet active. For larger test sizes, the advantage of SLMD-LightGBM becomes increasingly evident. Specifically, at day two, SLMD-LightGBM achieves a MAPE of 24.51% compared to LMD-LightGBM's 25.89%, while the MSE values are 110.78 and 117.70, respectively. This trend continues as the data set expands. The self-updating module proves effective as the test size increases, highlighting its ability to adapt to changing data patterns over time.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on August 30,2026 at 06:03:08 UTC from IEEE Xplore. Restrictions apply.

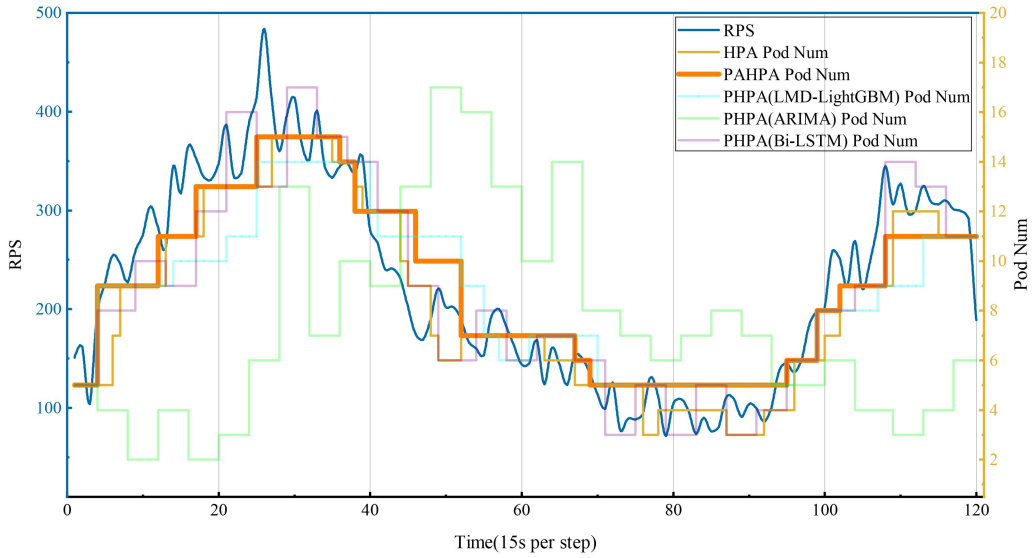


Fig. 11. The relationship between the number of pods and the workload over time.

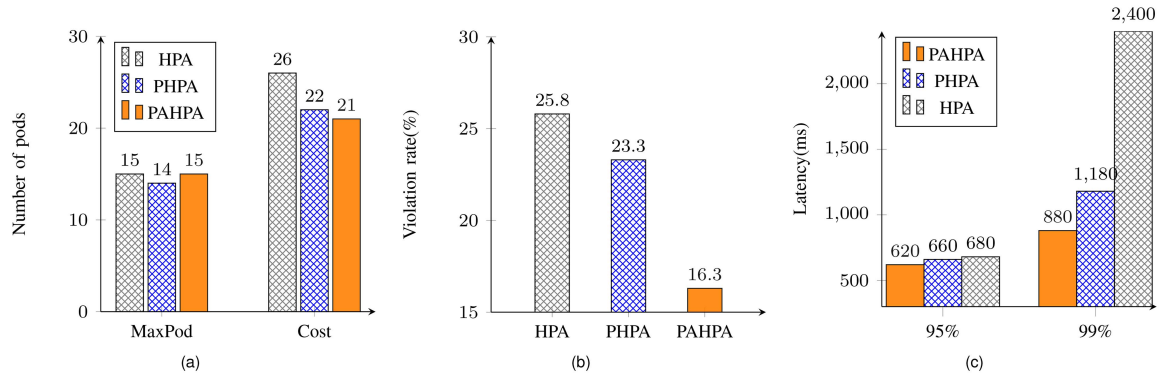


Fig. 12. MaxPod, Cost, Violation rate and Laterncy between different auto-scaling methods. (a) MaxPod and Cost between different auto-scaling methods. (b) Violation between different auto-scaling methods. (c) Latency between different auto-scaling methods.

These results validate the effectiveness of both the LMD decomposition and the self-updating mechanism in improving prediction accuracy and stability, particularly for long-term predicting tasks.

B. Scaling Phase

Fig. 11 illustrates the relationship between workload and pod scaling over time. The blue line represents the Queries Per Second (QPS), indicating workload fluctuations. The light-orange line corresponds to pods adjusted by the default reactive HPA, the orange line denotes the proposed PAHPA method, and the three dashed lines represent PHPA variants based on different prediction models: ARIMA (green), LMD-LightGBM (blue), and Bi-LSTM (purple).

Compared to reactive HPA, PAHPA scales pods earlier, as seen around timestamps 5, 25, and 100, due to its predictive capability. Moreover, during the scale-down phase, PAHPA maintains a higher number of pod replicas for a longer duration (e.g., between timestamps 40–100), allowing smoother request redistribution and thus better preserving QoS.

Among the PHPA variants, PHPA(ARIMA) exhibits noticeable lag in scaling decisions, reflecting its difficulty in capturing rapid workload changes. PHPA(Bi-LSTM) often triggers earlier scaling but tends to overestimate the required pod count, leading to occasional over-provisioning. In contrast, PHPA(LMD-LightGBM) delivers smoother pod count trajectories with fewer abrupt fluctuations. This behavior stems from the local smoothing effect of LMD decomposition, which attenuates highfrequency noise in the raw QPS signal, combined with LightGBM's ability to learn stable temporal patterns without overfitting. Consequently, LMD-LightGBM provides a more stable prediction baseline, which translates into steadier scaling decisions.

Overall, PAHPA which integrates predictive scaling with realtime correction via a maxselection strategy—achieves a more balanced tradeoff. It mitigates the lag inherent in purely reactive methods, tempers the overestimation tendency of some predictive models, and compensates for potential prediction underestimation by incorporating current load values. As a result,

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on August 30,2026 at 06:03:08 UTC from IEEE Xplore. Restrictions apply.

PAHPA maintains robust QoS while avoiding excessive resource waste, outperforming both the purely reactive HPA and the purely predictive PHPA variants across the considered metrics.

Fig. 12 presents a comparative analysis of PAHPA with default HPA and PHPA (HPA based on LMD-LightGBM specifically, which performs best among three PHPA methods.). In terms of MaxPod (Fig. 12(a)), both HPA and PAHPA reach a maximum of 15 pods, indicating their ability to handle peak QPS and maintain QoS. PHPA only reaches 11 MaxPod, likely due to prediction errors, demonstrating that scaling based solely on predictive metrics value is insufficient. Low MaxPod means there is no enough resources to handle high workload. Regarding overall cost, PAHPA requires only 21 pods to handle the workload, which is 5 fewer than HPA. PHPA uses just one more pod than PAHPA but doesn't match the workload effectively.

Fig. 12(b) illustrates the Violation Rate (VR) for each method. PAHPA achieves the best QoS with a VR of 16.3%, which is 9.5% lower than HPA (25.8%) and 8% lower than PHPA (23.3%). This indicates PAHPA's ability to maintain the desired performance threshold.

In Fig. 12(c), we analyze the latency performance across different percentiles for HPA, PHPA, and PAHPA. The measurements reveal consistent patterns in the latency values: at the 99th percentile, PAHPA records 880 ms, PHPA shows 1180 ms, and HPA reaches 2400 ms. Similar trends appear at other percentiles, with PAHPA maintaining 620 ms at the 95th percentile compared to PHPA's 660 ms and HPA's 680 ms. These results indicate better overall latency management for PAHPA across various load conditions.

VII. Conclusion

We have developed PAHPA, a novel Prediction-assisted Horizontal Pod Autoscaler for Kubernetes clusters. Our approach includes a newly proposed prediction model, SLMD-LightGBM, which combines Local Mean Decomposition, LightGBM, and a self-updating mechanism, demonstrates great prediction accuracy compared to other time series prediction methods, achieving a MAPE of 24.51%.

PAHPA uses real-time monitoring data to refine the predicted pod replicas, which reflects a central insight of this paper that predictions cannot be the sole basis for scaling decisions. PAHPA reduced the number of deployed pods compared to default HPA, leading to lower resource usage. It enhances system stability and reduces latency, achieving a 99.9th percentile latency of 880 ms compared to HPA's 2400 ms.

These results demonstrate the potential of our prediction-assisted approach in optimizing resource usage and performance in container orchestration systems. However, this paper has only discussed a simple deployment scenario with a single application. Future work is needed to explore the adoption of PAHPA in more complex and diverse microservice architectures.

Finally, we note that this work focuses solely on replica count decisions under standard Kubernetes load balancing. In large-scale or microservice-heavy environments, however, the interplay between autoscaling and load balancing becomes critical. Future work will explore co-designing predictive autoscaling with intelligent load distribution, where metrics such as request skew, pod utilization variance, or service mesh telemetry could inform more holistic scaling policies.

References

[1] "Kubernetes-homepage," kubernetes.io. Accessed: Apr. 12, 2024. [Online]. Available: https://kubernetes.io/

[2] C. Carrión, "Kubernetes scheduling: Taxonomy, ongoing issues and challenges," *ACM Comput. Surv.*, vol. 55, no. 7, pp. 1–37, Dec. 2022.

[3] C. Carrión, "Kubernetes as a standard container orchestrator-a bibliometric analysis," *J. Grid Comput.*, vol. 20, no. 4, 2022, Art. no. 42.

[4] M.-N. Tran, D.-D. Vu, and Y. Kim, "A survey of autoscaling in kubernetes," in *Proc. 13th Int. Conf. Ubiquitous Future Netw.*, 2022, pp. 263–265.

[5] B. Choi, J. Park, C. Lee, and D. Han, "PHPA: A proactive autoscaling framework for microservice chain," in *Proc. 5th Asia-Pacific Workshop Netw.*, 2022, pp. 65–71.

[6] O. P. Egbuna, "Machine learning applications in kubernetes for autonomous container management," *J. Artif. Intell. Res.*, vol. 4, no. 1, pp. 196–219, 2024.

[7] R. N. Calheiros, E. Masoumi, R. Ranjan, and R. Buyya, "Workload prediction using ARIMA model and its impact on cloud applications' QoS," *IEEE Trans. Cloud Comput.*, vol. 3, no. 4, pp. 449–458, Fourth Quarter 2015.

[8] M. Barati and S. Sharifian, "A hybrid heuristic-based tuned support vector regression model for cloud load prediction," *J. Supercomputing*, vol. 71, pp. 4235–4259, 2015.

[9] W. Zhong, Y. Zhuang, J. Sun, and J. Gu, "A load prediction model for cloud computing using pso-based weighted wavelet support vector machine," *Appl. Intell.*, vol. 48, pp. 4072–4083, 2018.

[10] F. Rossi, "Auto-scaling policies to adapt the application deployment in kubernetes," in *Proc. 12th ZEUS Workshop*, 2020, pp. 30–38.

[11] N.-M. Dang-Quang and M. Yoo, "Deep learning-based autoscaling using bidirectional long short-term memory for kubernetes," *Appl. Sci.*, vol. 11, no. 9, 2021, Art. no. 3835.

[12] X. Tang, Q. Liu, Y. Dong, J. Han, and Z. Zhang, "Fisher: An efficient container load prediction model with deep neural network in clouds," in *Proc. 2018 IEEE Int. Conf. Parallel Distrib. Process. Appl., Ubiquitous Comput. Commun., Big Data Cloud Comput., Social Comput. Netw., Sustain. Comput. Commun.*, 2018, pp. 199–206.

[13] J. Zhang, L. Guo, L. Song, S. Gao, C. Hao, and X. Li, "PatchTCN: Patch-based transformer convolutional network for times series analysis," in *Proc. 3rd Int. Symp. Comput. Artif. Intell.*, 2024, pp. 1–9.

[14] T.-T. Nguyen, Y.-J. Yeom, T. Kim, D.-H. Park, and S. Kim, "Horizontal pod autoscaling in kubernetes for elastic container orchestration," *Sensors*, vol. 20, no. 16, 2020, Art. no. 4621.

[15] L. Toka, G. Dobreff, B. Fodor, and B. Sonkoly, "Machine learning-based scaling management for kubernetes edge clusters," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 1, pp. 958–972, Mar. 2021.

[16] N. C. Coulson, S. Sotiriadis, and N. Bessis, "Adaptive microservice scaling for elastic applications," *IEEE Internet Things J.*, vol. 7, no. 5, pp. 4195–4202, May 2020.

[17] L. Baresi, D. Y. X. Hu, G. Quattrocchi, and L. Terracciano, "Kosmos: Vertical and horizontal resource autoscaling for kubernetes," in *Proc. Int. Conf. Serv.-Oriented Comput.*, 2021, pp. 821–829.

[18] J. Rahman and P. Lama, "Predicting the end-to-end tail latency of containerized microservices in the cloud," in *Proc. 2019 IEEE Int. Conf. Cloud Eng.*, 2019, pp. 200–210.

[19] W.-K. Lai, Y.-C. Wang, and S.-C. Wei, "Delay-aware container scheduling in kubernetes," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11813–11824, Jul. 2023.

[20] Z. Zhou et al., "Ahpa: Adaptive horizontal pod autoscaling systems on alibaba cloud container service for kubernetes," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 15621–15629.

[21] G. Yu, P. Chen, and Z. Zheng, "Microscaler: Cost-effective scaling for microservice applications in the cloud with an online learning approach," *IEEE Trans. Cloud Comput.*, vol. 10, no. 2, pp. 1100–1116, Second Quarter 2020.

[22] Q. Wen, K. He, L. Sun, Y. Zhang, M. Ke, and H. Xu, "Robustperiod: Robust time-frequency mining for multiple periodicity detection," in *Proc. 2021 Int. Conf. Manage. Data*, 2021, pp. 2328–2337.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on August 30,2026 at 06:03:08 UTC from IEEE Xplore. Restrictions apply.

[23] R. He, L. Zhang, and A. W. Z. Chew, "Modeling and predicting rainfall time series using seasonal-trend decomposition and machine learning," *Knowl.-Based Syst.*, vol. 251, 2022, Art. no. 109125.
[24] Y. Chang et al., "Rail crack detection using optimal local mean decomposition and cepstral information coefficient based on electromagnetic acoustic emission technology," *IEEE Trans. Instrum. Meas.*, vol. 73, 2024, Art. no. 9506412.
[25] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 3149–3157.
[26] D. Sirisha and S. S. Prasad, "MPEFT: A makespan minimizing heuristic scheduling algorithm for workflows in heterogeneous computing systems," *CCF Trans. High Perform. Comput.*, vol. 5, no. 4, pp. 374–389, 2023.
[27] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Inf. Fusion*, vol. 81, pp. 84–90, 2022.
[28] L. Li, Z. Liu, J. Shen, F. Wang, W. Qi, and S. Jeon, "A lightgbm-based strategy to predict tunnel rockmass class from TBM construction data for building control," *Adv. Eng. Informat.*, vol. 58, 2023, Art. no. 102130.
[29] Z. Ding, S. Wang, and C. Jiang, "Kubernetes-oriented microservice placement with dynamic resource allocation," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1777–1793, 2022.
[30] Prometheus-from metrics to insight, prometheus.io. Accessed: Apr. 12, 2024. [Online]. Available: https://prometheus.io/
[31] J. H. Dshalalow, *Advances in Queueing Theory, Methods, and Open Problems*. Boca Raton, FL, USA: CRC Press, 2023.
[32] "Locust-an open source load testing tool," locust.io. Accessed: Apr. 12, 2024. [Online]. Available: https://locust.io/



Junwei Xiao is currently working toward the MS degree in Software Engineering from Central South University of Forestry and Technology, Changsha, China. His research interests include microservice and cloud computing.



Fan Yang received the PhD degree in computer science and engineering from Hunan University, China, in 2016. He is currently an associate professor with Central South University of Forestry and Technology. He was a visit scholar with Michigan State University, US, from 2014-2015. His research interests include Cyber Physical Systems, Embedded Systems and Edge Computing.



Wei Ai received the PhD degree from the College of Computer Science and Mathematics, Hunan University, Changsha, China. Her research interests include date mining, Big Data, cloud computing, and parallel computing.



Tao Meng received the PhD degree from the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China, in Jan. 2020. He is currently a lecturer with the College of Computer Science and Mathematics, Central South University of Forestry and Technology, Changsha, China. His research interests include graph neural networks, multimodal fusion, and edge computing.



Guoqi Xie (Senior Member, IEEE) received the PhD degree in computer science and engineering from Hunan University, China, in 2014. He is a professor of computer science and engineering with Hunan University and the director of the Key Laboratory for Embedded and Network Computing of Hunan Province. He was a postdoctoral research fellow with Nagoya University, Japan. He is the founder and leader of ZVM (Zephyr-based Virtual Machine), which is a new-generation Type 1.5 embedded virtualization solution (Hypervisor). His current research interests include real-time systems, embedded virtualization, time-sensitive networking, system safety and security. He is currently serving on the secretary-general of the CCF Technical Committee on Embedded Systems and the editorial boards of the Journal of Systems Architecture, the Microprocessors and Microsystems, and the Journal of Circuits, Systems, and Computers. He is an ACM senior member.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY SURATHKAL. Downloaded on August 30,2026 at 06:03:08 UTC from IEEE Xplore. Restrictions apply.